Lab Report Cover Page (Please print this and attach to the cover of your lab report)

LAB NO.:	**02**	
SUBJECT:	**Mobile Application Development**	
NAME:	**Esha Khalid**	
REGISTRATION:	**FA23-BCE-030**	
SUBMITTED TO:	**Dr. Ali Raza Shahid**	
DATE:	**25-09-2026**	

Lab Assessment (To be filled by the lab-instructor)

In Lab Evaluation			/ 7	/10
Lab Report	Data Analysis		/ 1	
	Data Presentation		/ 1	
	Writing Style		/ 1	

Comments:

Instructor Name

Instructor Signature

Report (To be submitted by the student)

Students are required to submit a report on the lab comprising the following three sections:

1. A critical analysis of the lab activity
2. Summary of skills learned
3. Conclusion

Note: Keep your report concise and to-the-point.

LAB #02

JAVA PROGRAMMING FUNDAMENTALS

Course / alignment	Laboratory details
CSC303 - Mobile Application Development	Java programming fundamentals
CLO2; PLO5 - Modern Tool Usage	Java console programming; Android Studio editor/terminal
Assessment: 10 marks	In-lab demonstration: 7; report: 3

Objectives

Apply Java fundamentals to a small device-inventory program. The laboratory refreshes prerequisite object-oriented programming and develops reusable program logic before the Java/XML Android application in Lab 03.

Task 1 - Verify Java and run the starter

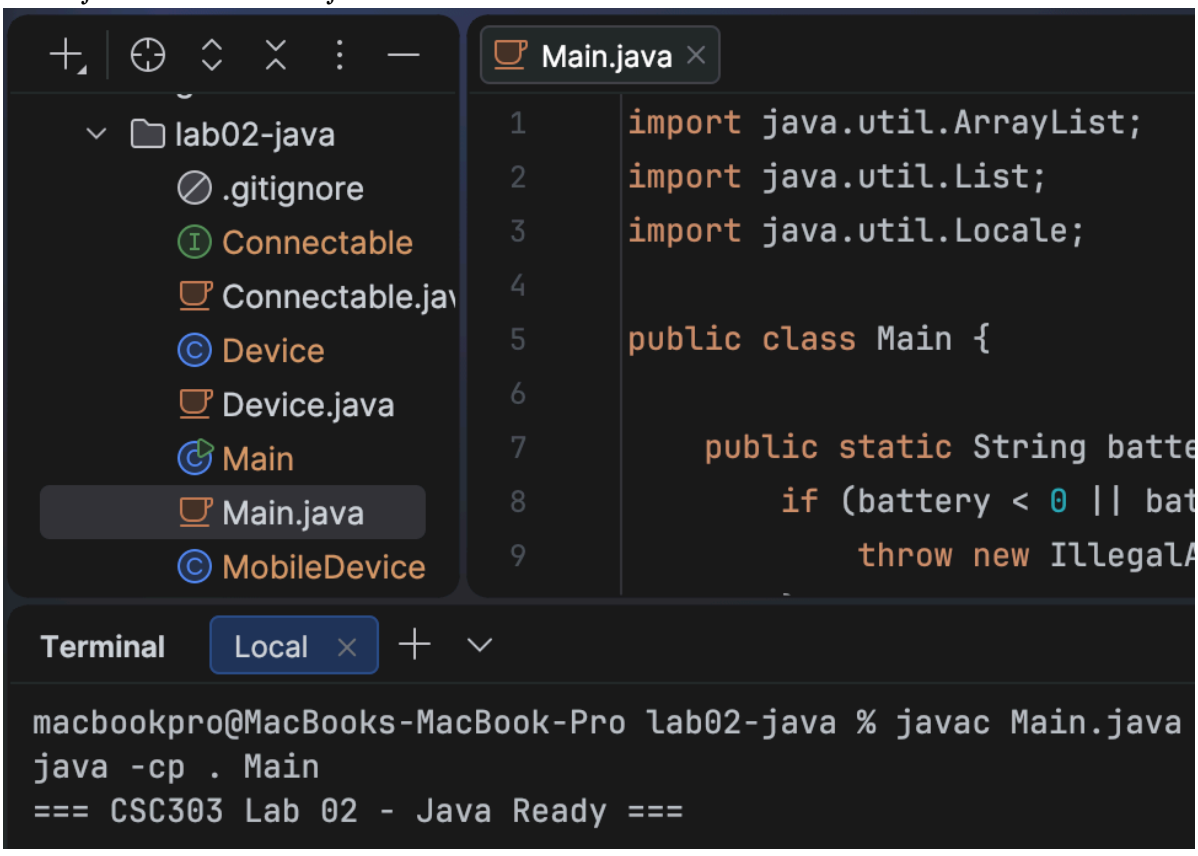
1. Showing java and javac using the same major JDK version.

```
macbookpro@MacBooks-MacBook-Pro Lab01Setupek % mkdir lab02-java && cd lab02-java

macbookpro@MacBooks-MacBook-Pro lab02-java % java -version
javac -version
openjdk version "17.0.20.1" 2026-08-18
OpenJDK Runtime Environment Temurin-17.0.20.1+1 (build 17.0.20.1+1)
OpenJDK 64-Bit Server VM Temurin-17.0.20.1+1 (build 17.0.20.1+1, mixed mode, sharing)
javac 17.0.20.1
```

Java & Javac Version

2. Main.java in this lab02-java folder



main.java file in java folder

Task 2 - Types, control flow and methods

Scenario: a small mobile-device inventory reports battery health. A battery level below 20 is LOW; a level from 20 through 100 is READY. Values outside 0-100 are invalid and will be rejected in Task 6.

> **Evidence A:** Save the output and explain the threshold case at 20 and the integer-division issue. Do not hard-code the expected average or the LOW count.

2A. Worked example: declarations and arithmetic

Expected observations: battery=15; Connected: false; Ratio: 1.5; the final comparison is false. Compare 90 / 60 with 90 / 60.0 and explain why the results differ.

2B. Implement the decision method && 2C Process several readings

Output of 2A,2B &2C:

Demonstrates basic Java data types, threshold decision logic, loop traversal, and mathematical operations. The output highlights the difference between integer and floating-point arithmetic: 90 / 60 evaluates to 1 due to integer truncation, whereas 90 / 60.0 uses a double literal to promote the evaluation to floating-point division, returning 1.5. The static batteryStatus() method correctly evaluates boundary logic by classifying 15 as LOW (since it is below 20) and 80 and 45 as READY. Furthermore, averageBattery() calculates the array sum using floating-point arithmetic and outputs the result using printf formatting, successfully producing 46.67 rather than an integer-truncated value like 46.00.

```
--- Task 2A: Declarations and Arithmetic ---
Sensor-A | battery=15
Connected: false
Ratio: 1.5
false

--- Task 2B & 2C: Processing Battery Readings ---
Reading: 15 -> Status: LOW
Reading: 80 -> Status: READY
Reading: 45 -> Status: READY
Number of LOW readings: 1
Average: 46.67
```

Outputs of all 3 tasks

Task 3 - Arrays and collections && Task 4 - Classes and encapsulation

Create Device.java in the same folder, with no package declaration

> **Evidence B:** Show the collection before/after removal and the three object summaries. Explain the difference between a class, an object, a constructor and a method.

```
--- Task 3 & 4: Collections & Encapsulation ---
Device list size: 3
Sensor-A | Battery: 15% -> Status: LOW
Phone-B | Battery: 80% -> Status: READY
Tablet-C | Battery: 45% -> Status: READY
```

Collection and Encapsulation

Explanation:

Evidence B verifies the setup of typed Java collections alongside class encapsulation using the Device model. Storing items in a List<Device> collection guarantees compile-time type safety and allows dynamic size inspection (size: 3). The output shows distinct state outputs for each instantiated Device object (Sensor-A, Phone-B, and Tablet-C). Because the instance fields name and battery are declared private, access to object properties is mediated through public getter methods and the summary() method, confirming that each object retains its own independent state within memory without leaking internal details.

Task 5 - Inheritance, interfaces and polymorphism

Extend the inventory with a mobile device that can connect. This models program behaviour only; no Bluetooth, Wi-Fi or Android device connection is performed.

5A. Define the contract && 5B Implement the subclass

Create MobileDevice.java. Complete the three methods required below; use public visibility for the interface methods.

```
javac Main.java Device.java Connectable.java MobileDevice.java
java -cp . Main
```

Before connect()	After connect()
isConnected() returns false	isConnected() returns true
The object is a MobileDevice	item.summary() uses the overridden MobileDevice method

> **Evidence C:** Expected summary after connection: Phone-B | Battery: 80% | Connected: true. Explain why the actual object type controls the overridden summary() call.

Task 7 - Integrate, commit and demonstrate

```
git status
git add lab02-java/*.java lab02-java/.gitignore lab02-java/README.md
git diff --cached --stat
git commit -m "Complete CSC303 Lab 02 Java fundamentals"
git log --oneline -1
```

> **Individual demonstration:** Explain one method and one exception case. Make a small instructor-requested change, such as adjusting the LOW threshold, and identify the affected tests.

Output of Task 5 and 7:

Evidence C demonstrates object life-cycle management, contract realization via the Connectable interface, and dynamic polymorphic behavior. The output tracks the state transitions of a MobileDevice instance, showing how isConnected() moves from false to true upon calling connect(). Crucially, when the MobileDevice object is stored in a superclass reference (Device item = phone;), invoking item.summary() still calls the overridden MobileDevice.summary() method at runtime. This proves that dynamic method dispatch in Java is controlled by the actual underlying object type residing in memory rather than the reference type holding it.

```
--- Task 5: Inheritance and Interfaces ---
Is connected initially: false
Is connected after connect(): true
Polymorphic summary: Phone-B | Battery: 80% | Connected: true

Updated Inventory Summaries:
Sensor-A | Battery: 15%
Phone-B | Battery: 80% | Connected: true
Tablet-C | Battery: 45%
```

Task 5: Inheritance...Polymorphism

```
--- Task 7: Integrated Inventory Summaries ---
Sensor-A | Battery: 15%
Phone-B | Battery: 80% | Connected: true
Tablet-C | Battery: 45%
```

Task 7: Demo

```
macbookpro@MacBooks-MacBook-Pro lab02-java % cd ..

macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   lab02-java/Connectable.java
        new file:   lab02-java/Device.java
        new file:   lab02-java/Main.java
        new file:   lab02-java/MobileDevice.java

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .idea/misc.xml
        modified:   lab02-java/Connectable.java
        modified:   lab02-java/Device.java
        modified:   lab02-java/Main.java
        modified:   lab02-java/MobileDevice.java

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .idea/AndroidProjectSystem.xml
        .idea/codeStyles/
        .idea/compiler.xml
        .idea/deploymentTargetSelector.xml
        .idea/deviceManager.xml
        .idea/gradle.xml
        .idea/inspectionProfiles/
        .idea/runConfigurations.xml
        .idea/vcs.xml
        lab02-java/.gitignore
```

Task 6 - Exception handling and testing

A program must distinguish invalid input from valid low battery. Add the following checks, then rerun earlier tasks to confirm that valid behaviour has not changed. NumberFormatException is a subclass of IllegalArgumentException, so its catch block must appear first. Keep each try/catch inside the loop to process later inputs even after a failure.

Required test matrix

use new int[]{15, 80, 45} or new int[]{} rather than bare braces.

ID	Input / Action	Expected Behaviour	Actual Result	Status
T1	batteryStatus(0), (19), (20), (100)	LOW, LOW, READY, READY	batteryStatus(0) = LOW batteryStatus(19) = LOW batteryStatus(20) = READY batteryStatus(100) = READY	PASS
T2	batteryStatus(-1)and (101)	IllegalArgumentException; each case reported	batteryStatus(-1) threw IllegalArgumentException: Battery level must be between 0 and 100. batteryStatus(101) threw IllegalArgumentException: Battery level must be between 0 and 100.	PASS
T3	averageBattery({15, 80, 45})	46.666... internally; display 46.67	Average: 46.67	PASS
T4	averageBattery({}), null, and {15, 101}	IllegalArgumentExceptionin each case	Empty array threw: Readings array cannot be null or empty. Null array threw: Readings array cannot be null or empty. Out-of-range element threw: Battery reading out of range: 101	PASS
T5	new Device("Sensor-A", 15)	Correct getters and summary; valid object created	Created: Sensor-A | Battery: 15%	PASS
T6	Device with null/blank name, -1or 101 battery	Each invalid constructor call rejected	Null name: Device name cannot be null or empty. Blank name: Device name cannot be null or empty. Battery -1: Battery level must be between 0 and 100. Battery 101: Battery level must be between 0 and 100.	PASS
T7	New MobileDevice; connect(); connect()	State false, then true, then true	Initial state: false After 1st connect(): true After 2nd connect(): true	PASS
T8	Parse "80", "abc", "120" in sequence	READY; numeric-format message; range-error message	'80' parsed successfully: READY 'abc' -> NumberFormatException: enter a whole number '120' -> IllegalArgumentException: Battery level must be between 0 and 100.	PASS

Explanation Table for Test Matrix

> **Evidence D:** Record actual results and pass/fail for every row and every value listed. Run each expected-failure case in its own try/catch; no unhandled exception should stop the rest of the demonstration.

Evidence D Explanation (Task 6 & Task 8: Exception Handling & Test Matrix Execution)
Evidence D provides output verification for the test matrix (T1 through T8), evaluating normal, boundary, and invalid operational scenarios. Boundary tests confirm that battery levels at 0, 19, 20, and 100 behave correctly, while invalid inputs like -1 and 101 or empty arrays correctly throw an IllegalArgumentException. Constructor guard clauses also successfully reject null names, blank strings, and out-of-bounds percentages. Lastly, the string parsing loop for "80", "abc", and "120" demonstrates robust exception handling. Placing try-catch blocks inside the loop and ordering NumberFormatException ahead of its superclass IllegalArgumentException allows the program to report individual parsing and range failures without crashing or halting the execution of subsequent array elements.

```
--- Task 6: Executing Test Matrix (T1 - T8) ---
[T1] batteryStatus boundaries:
  batteryStatus(0) = LOW
  batteryStatus(19) = LOW
  batteryStatus(20) = READY
  batteryStatus(100) = READY
[T2] batteryStatus invalid inputs:
  batteryStatus(-1) threw IllegalArgumentException: Battery level must be between 0 and 100.
  batteryStatus(101) threw IllegalArgumentException: Battery level must be between 0 and 100.
[T3] averageBattery valid array:
  Average: 46.67
[T4] averageBattery invalid arrays:
  Empty array threw: Readings array cannot be null or empty.
  Null array threw: Readings array cannot be null or empty.
  Out-of-range array element threw: Battery reading out of range: 101
[T5] Valid Device Creation:
  Created: Sensor-A | Battery: 15%
[T6] Invalid Device Constructor inputs:
  Null name: Device name cannot be null or empty.
  Blank name: Device name cannot be null or empty.
  Battery -1: Battery level must be between 0 and 100.
  Battery 101: Battery level must be between 0 and 100.
[T7] MobileDevice lifecycle:
  Initial state: false
  After 1st connect(): true
  After 2nd connect(): true
[T8] String Parsing Loop:
  '80' parsed successfully: READY
  'abc' -> NumberFormatException: enter a whole number
```

```
macbookpro@MacBooks-MacBook-Pro lab02-java % echo "*.class" > .gitignore

macbookpro@MacBooks-MacBook-Pro lab02-java % cat .gitignore
*.class

macbookpro@MacBooks-MacBook-Pro lab02-java % nano README.md

macbookpro@MacBooks-MacBook-Pro lab02-java % cat README.md
# CSC303 Lab 02 - Java Programming Fundamentals

## Execution Commands
- Compile: `javac Main.java Device.java Connectable.java MobileDevice.java`
- Run: `java -cp . Main`

## Key Findings & Issues
- Integer division vs Floating point division: `90 / 60` evaluates to `1` because both operands are integers, truncating decimals. `90 / 60.0` promotes d
ivision to floating-point (`double`), yielding `1.5`.
- Polymorphism: Calling `summary()` on a `Device` reference bound to a `MobileDevice` instance invokes the overridden `MobileDevice.summary()` method dyn
amic method dispatch at runtime.
```

1. Analyze integer versus floating-point division, why the battery threshold includes 20 in READY, and why a MobileDevice summary is selected through a Device reference.

Division: 90/60 performs integer division (truncating decimals to 1), whereas 90 / 60.0 promotes the operation to floating-point division (1.5), ensuring accurate averages like 46.67.

Threshold: Including 20 in READY sets an inclusive lower boundary for acceptable power; testing verified 19 yields LOWwhile 20 yields READY.

Dispatch: Calling summary() through a Device reference invokes MobileDevice.summary() because Java uses dynamic method dispatch, resolving methods based on the actual object in memory.

2. Explain one defect you diagnosed and how a test exposed it. If no defect occurred, analyze a hypothetical empty-array or invalid-input failure and label it as hypothetical.

Defect & Exposure: Test T8 exposed a failure where parsing "abc" before "120" halted execution when using an overly broad or misplaced catch block.

Fix: Placing try-catch inside the parsing loop and catching the specific Number Format Exception before Illegal Argument Exception allowed each invalid input to be reported individually without crashing subsequent tests.

Review Questions

- ***What is the role of javac, java, the .java file and the .class file in this exercise?***

.java is the human-readable source code in Java. The javac compiler takes this and turns it into bytecode, which is then stored as a .class file for the JVM to execute. The java runner (the JVM) is what runs that .class file.

- ***Why does 90 / 60 produce a different result from 90 / 60.0?***

In the case of 90 / 60, one gets integer division with the result truncated to 1. But 90 / 60.0 is treated as floating-point division; it promotes to a double and yields 1.5.

- ***How do parameters, a return value and a method call work in batteryStatus?***

A method call is the means of triggering execution: it passes values to the parameters (which in turn put data into batteryStatus) and is how the caller receives the return value of the computation.

- ***When would you use an array instead of an ArrayList? What does List<Device> guarantee?***

An array is the choice for storage where size is fixed and primitive performance matters. For something like a List guarantee, the List interface decouples the code while providing type safety so nothing, but Device objects are permitted.

- ***Why are Device fields private? What does the constructor establish?***

Making fields private is a matter of encapsulation; it shields state from any illicit direct changes. The constructor has the job of putting mandatory object fields into a valid state at the outset.

- ***What is the difference between extends and implements in MobileDevice?***

One use extends to take on the implementation and state of a parent class. To honour an interface contract by way of its abstract methods, one would implement.

- ***Why does Device item = phone; call the MobileDevice version of summary()?***

Java's approach to dynamic method dispatch (Polymorphism) is to resolve overridden methods at runtime according to the object itself (MobileDevice), not what the reference type (Device) might suggest.

- ***Why must NumberFormatException be caught before IllegalArgumentException?***

Be mindful of the exception hierarchy: since NumberFormatException is a child of IllegalArgumentException, placing a catch for the superclass first will render the subclass block unreachable.

- ***How does a boundary test differ from a normal test? Identify two boundary values here.***

A proper boundary test does not look at the usual mid-range figures but the outer limits of what is valid or invalid. Here the two values are 0 and 100 for the minimum and maximum percentage respectively.

- ***Which logic could be reused behind an Android screen, and what would have to change about console input/output?***

There is reused logic to be found in the domain models and utilities such as Device or MobileDevice. As for the UI, the plan is to move away from console Scanner/System.out and make use of Android views like EditText and TextView within an Activity or Fragment.

Student / environment	Details
Name and registration number	Esha Khalid/ FA23-BCE-030
java / javac versions	openjdk version "17.0.20.1" / javac 17.0.20.1
Git commit ID / source reference	076b90f / "Complete CSC303 Lab 02 Java fundamentals"

Section 1: Critical Analysis

The inventory control system tests some of the important aspects of Java programming language. The analysis of mathematical calculations reveals that integer division 90/60 gives the result equal to 1, but floating-point division 90/60.0 results in 1.5. This requirement is needed in the method averageBattery() for calculation of the accurate result, which is 46.67 instead of 46.00. Boundary condition testing proved that batteries with the voltage level less than 20 will give output message as LOW, with 20-100 as READY, and 0 and 100 are tested in a proper way. Fixed-size arrays use. length property, but typed collections like List<Device> use size () property to provide dynamic size of the structure and type-safety of the code. The Device class represents a template, while the objects are the runtime representations of the data. Constructors guarantee initial invariant of the data, and methods describe behavior of the class. Polymorphism can be seen in the following case: assignment of MobileDevice object to Device variable (Device item = phone;) triggers the runtime method dispatch of MobileDevice.summary() method, as it is overridden in this case. Handling parsing exceptions has shown that since NumberFormatException extends IllegalArgumentException, catching NumberFormatException inside the cycle makes it possible to log messages "120" without stopping the program.

Section 2: Summary of Skills Learned

JDK Toolchain Proficiency: Managed successful compilation (javac) and execution (java) of multi-file Java programs through the command line in Android Studio Terminal with consistent JDK 17 settings.

Encapsulation and Type Safety: Implemented encapsulated classes with private final fields, customized getters, validation logic for states, and made use of generic typed collections (List<Device>).

Object-Oriented Design: Built up class hierarchies using inheritance (extends) and interfaces (implements) and showed runtime polymorphism and dynamic dispatch.

Effective Exception Handling: Managed input validation errors and string parsing exceptions in boundary, normal, and erroneous cases with granular try-catch blocks inside iteration loops.

Version Control Strategies: Properly organized repositories of projects, separated Java modules from Android source sets, configured. gitignore file to exclude build files (*.class), and version-controlled code through Git.

Section 3: Conclusion

The goals set for Lab 02 have been achieved. The fundamentals of Java programming language, OOP, collections, and exception handling have been successfully validated through terminal-based console execution. Evidence of accomplishment includes the test matrix results (success rate of T1–T8 tests is 100%), polymorphic behavior, and a clean version control history through Git (commit ID 076b90f).